Static Count-One Ownership for In-Place Array Updates in Lean 4

[Author]

[Date]

Gray-ruled blocks mark prose positions; the Lean statements are quoted from the development, with hypotheses abbreviated where stated.

Abstract

> *One paragraph: the problem (Lean arrays copy when shared), the claim (a count-one ownership discipline yields in-place writes for the recognised emitted write), and the boundary (proved in the model and runtime model; measured on the linked corpus; trusted at the C primitives).*

Argument map

> *Introduce the spine of the paper: each row is a step of the argument and the claim that carries it.*

Step	Claim	Carrier
1	accepted writes are exclusive	Claim 2
2	the entry invariant supplies count one and separation	Claim 3
3	the emitted write prefix is recognised and corresponds	Claim 4
4	the recognised write lands in place	Claim 5
5	the nested path corresponds without a callee hypothesis	Claim 6
6	the extraction pair returns to count one	Claim 8
7	the put-back's exclusive branch is derived	Claim 9
8	the runtime model cannot abort under the correspondence	Claim 10
9	the linked behaviour agrees on a corpus	Section 8

1 Introduction

> *Motivate the gap: a purely functional language's updates copy when the array is shared; can a static discipline guarantee the in-place branch? State the two claims and the boundary. Cite [4, 12, 17].*

> *Contributions, each pointing at a claim; state the non-claims early.*

2 Setting: Lean 4, LCNF, and reference counting

> *The pinned toolchain, LCNF, the runtime's `lean_is_exclusive` and `lean_array_uset`, and array representation. Cite [4, 16, 12].*

State the problem: the runtime copies unless the count is one and the object is non-persistent.

3 The ownership discipline

3.1 The judgement and its rules

Introduce the context and the judgement: a context records each variable's use, reference and multiplicity; `check` (and `checkN` with the frame stack) returns the successor context or refuses. The rules below are ownership obligations, not runtime tests; the heap side is `Inv` and `EntryInv`.

Rule	Obligation	Effect
`let x (checkAlias a)`	`a` owned, multiplicity one	`x` is the checked alias
`setArr x i v`	`x` owned, multiplicity one; `i`, `v` scalar	exclusive write through `x`
`inc x n`, `dec x n`	`x` owned	multiplicity $\pm n$; `dec` erases
`takeOut a i fst snd e`	`a` owned, multiplicity one; `i` scalar; `e` multiplicity one; `fst`, `snd` dead and distinct; `e` ≠ `a`; no open frame at `a`	push the frame; bind parent and element
`putBack a e i result`	`a`, `e` owned, multiplicity one; the frame is `(a,i,e)`; `result` dead	pop the frame; bind the result
`callAt state path x result body`	`state` owned, multiplicity one; `path`, `x` scalar; `result` dead; `CountFree body`; no open frame at `state`; the body is accepted from the isolated context and returns `state`	call the body; bind the result
`ret x`	no frame: `x` owned or scalar; one frame: `x` is the frame's element; deeper: refused	exit

Explain the frame stack and the exit rule, and why the take-out and call rules are the ones that make nested updates safe. Note what the checker cannot see: slot contents, heap counts and client separation are supplied by the invariant and the fixture.

3.2 Worked examples

Each example is a `Code`/`NCode` value; the `#guard` is the executable check. The contexts and heaps are the repository fixtures: `initCtx` for the array and the nested `ctx0` for the paths.

Example 1 (A unique write).

```
def execSet (x y i v : Var) : Code :=
  .let y (.checkAlias x) (.setArr y i v (.ret y))

#guard (check (initCtx 0) (execSet 0 3 1 2)).isSome
```

Example 2 (An aliased write is refused).

```
def aliasSet (x y i v : Var) : Code :=
  .inc x 1 (execSet x y i v)

#guard (check (initCtx 0) (aliasSet 0 3 1 2)).isNone
```

The alias increments the handle, so the write's multiplicity-one obligation fails. The held context with multiplicity two is the same refusal at the context level.

Example 3 (An extraction and its return).

```
def good : NCode :=
  .takeOut 0 1 2 3 1 (.ret 3)

#guard (checkN ctx0 good).isSome
```

Example 4 (A stale slot is refused).

```
def bad : NCode :=
  .takeOut 0 1 2 3 1 (.takeOut 2 1 4 5 1 (.ret 5))

#guard (checkN ctx0 bad).isNone
```

The second extraction reopens the array whose slot the first has already emptied; the frame stack records the first extraction.

Example 5 (A recursive body confined to its state).

```
def good : NCode :=
  .takeOut 0 1 2 3 1
    (.callAt 3 4 7 9 (.ret 3) (.ret 9))

#guard (checkN ctx0 good).isSome
```

Example 6 (A suspended parent is refused).

```
def bad : NCode :=
  .takeOut 0 1 2 3 1
    (.callAt 3 4 7 9
      (.takeOut 2 4 5 6 2 (.putBack 5 6 4 8 (.ret 3)))
      (.ret 9))

#guard (checkN ctx0 bad).isNone
```

The refused body returns the call's own state, exactly as the accepted body does; the refusal is isolation: the suspended parent is not in the body's context.

3.3 The invariant and soundness

The model heap, counts, exclusivity and the checker as a proof obligation, not a runtime test. Count one is a ghost predicate preserved by the invariant.

Claim 1 (Invariant and exclusivity).

```
structure Inv (H : Heap) (env : Env) (c : Ctx) : Prop where
  rc_eq         : ∀ r, H.rc r = c.mult r
  owned         : ∀ x r, c.us x = .owned → c.refOf x = some r →
                    env x = some (.ref r) ∧ 1 ≤ c.mult r
  borrow        : ∀ x r, c.us x = .borrow → c.refOf x = some r →
                    env x = some (.ref r) ∧ H.cells r ≠ none
  scalar        : ∀ x, c.us x = .scalar → ∃ n, env x = some (.scalar n)
  nonPersistent : ∀ r o, H.cells r = some o → o.persistent = false
```

```
def Exclusive (H : Heap) (r : Ref) : Prop :=
  ∃ o, H.cells r = some o ∧ o.rc = 1 ∧ o.persistent = false
```

Claim 2 (Checker soundness).

```
theorem check_sound {c : Code} {ctx ctx' : Ctx} {H : Heap} {env : Env}
    (hc : check ctx c = some ctx') (hi : Inv H env ctx) :
    ∃ H' env' v ws,
      run c env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧ Inv H' env' ctx'
```

Claim 3 (Entry invariant).

```
structure EntryInv (H : Heap) (env : Env) (c : Ctx) (root : Ref)
    (client : List Ref) : Prop extends Inv H env c where
  uniqueTree : UniqueTree H root
  separated  : ClientSeparated H root client
  complete   : RootsComplete c H root
```

```
theorem checkN_sound_of_entryInv
    (hEntry : EntryInv H env c root client)
    (hag : Agrees H env code) (hc : checkN c code = some c') :
    ∃ H' env' v ws,
      runN code env H = .done H' env' v ws ∧
      (∀ w ∈ ws, w.2 = true) ∧ Inv H' env' c'
```

Hypotheses abbreviated: the statement is the repository's, with the heap/environment parameters implicit.

Path-level agreement and annotation reachability replace per-client premises: `agrees_of_path`, `annotationsIn_of_path`, the transitivity lemmas, and `path_run_safe`, and `client_retention`. State what remains supplied.

4 Recognising the emitted write

The strict-recogniser approach: decode the actual LCNF value, pin operation order, arities, operand identities and branch constructors, and reject token-preserving near misses. The recogniser is structural; A1–A12 (execP) and N1–N12 (nested) are named assumptions.

The three shapes: `execP`'s `PTree.set`, `setAt`'s `Path.index`, and the `Path.left`/`Path.right` pair arms with joins. Near-miss counts: 11, 10, 28.

5 Correspondence to the ownership model

The semantic bridge: the recognised fragment's restricted semantics equals the model run; recognition is load-bearing because `execAST` returns nothing unless the code decodes.

Claim 4 (Prefix correspondence).

```
theorem project_sound (assign : FVarId → Var) (p : Prefix)
    (code : Code .impure) (hproj : projectPrefix code = some p)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
      some (PrefixOutcome.ofModel (Ownership.run (p.toModel assign) env H))
```

Hypotheses abbreviated: layout and the heap/environment parameters.

Claim 5 (The write is in place).

```
theorem recognized_write_in_place
    (hproj : projectPrefix code = some p) (hl : Layout assign p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    PrefixOutcome.ofExec assign p (execAST assign p code s) =
        some (inPlaceOutcome root i v o H) ∧
    PrefixOutcome.ofModel (Ownership.run (p.toModel assign) env H) =
        inPlaceOutcome root i v o H
```

Hypotheses abbreviated: the machine, environment and heap parameters.

The nested path: `build` nests take-out/call/put-back per level; the finite-path induction discharges the callee contract from the tail, leaving the leaf contract and N10. The flat environment cannot express the emitted per-invocation `oproj[1]` index; record that bound.

Claim 6 (Nested finite path).

```
theorem index_path_simulates_model
    (declName takeOut : Name) (assign : FVarId → Ownership.Var)
    (p : NestedPrefix) (code : Code .impure)
    (hproj : projectNested declName takeOut code = some p)
    (ls : List Level) :
    NestedOutcome.ofExec assign p
        (execNestedAST declName takeOut assign body p code s) =
      NestedOutcome.ofModel (Nested.runN (build path value body ls) mEnv H) ∧
    ∃ H' env' ws,
      Nested.runN (build path value body ls) mEnv H =
        .done H' env' (.ref rA) ws ∧ (∀ w ∈ ws, w.2 = true)
```

Hypotheses abbreviated: the level facts, agreement, entry state and the leaf contract.

Claim 7 (The finite nest).

```
theorem calls_build (path value : Var) (body : NCode) (ls : List Level) :
    calls (build path value body ls) = calls body + ls.length
```

```
theorem callBody_build (path value : Var) (body : NCode) (l : Level)
    (rest : List Level) :
```

```
    callBody? (build path value body (l :: rest)) =
      some (build path value body rest)
```

6 Counted extraction, call, and restoration

The pair's temporary handles: the take-out transfers the parent and child into the fresh pair, the projections increment, and freeing the unique pair releases both edges. The negative controls make the claim non-vacuous.

Claim 8 (Pair accounting).

```
theorem pair_accounting {H : Heap} {p c : Ref} {i : Nat}
    (hw : WellFormedEntry H p c i) :
    let B := protocol H p c i
    B.2.rc p = 1 ∧ B.2.rc c = 1 ∧
      B.2.cells p ≠ none ∧ B.2.cells c ≠ none ∧
      B.2.cells B.1 = none ∧
      Heap.Exclusive B.2 p ∧ Heap.Exclusive B.2 c ∧
      B.2.rc p = H.rc p ∧ B.2.rc c = H.rc c
```

```
theorem no_free_components_not_exclusive
    (hw : WellFormedEntry H p c i) :
    let B := protocolNoFree H p c i
    B.2.rc p = 2 ∧ B.2.rc c = 2 ∧ ¬ Heap.Exclusive B.2 p
```

Put-back: two dispositions; the exclusive branch is derived from the counted-pair boundary and the child-call boundary. Functional restoration and count preservation are separate; the returned-handle counts remain premises.

Claim 9 (Put-back exclusivity).

```
theorem putBack_exclusive_of_wellFormed
    (hw : CountedPair.WellFormedEntry H p c i)
    (hc : ChildCall Hin Hout env ctx state path x body p c) :
    ∃ H',
      putBack H p i c = some (p, H') ∧
      H'.cells p = (mutate H p o i c).cells p ∧
      Heap.Exclusive H' p ∧ H.rc p = 1
```

Hypotheses abbreviated: the child's admitted body and the returned-handle accounting; the conclusion is the exclusive branch and the parent's count.

7 Runtime model and abort impossibility

The independent runtime state, its exclusivity predicate and both array-set dispositions; the structural correspondence preserves refs, cells, counts, persistence, fields and the allocator cursor. The abort branch is unreachable under the correspondence and `EntryInv`; a guarded allocator refuses a dirty cursor. Name the fidelity boundary.

Define the runtime state, `isExclusive`, the check's ok/abort outcomes, the two array-set dispositions and the structural correspondence `Corr`; the definitions are independent of the ownership model's predicates.

Claim 10 (No abort).

```
theorem checkExclusive_no_abort {H : Heap} {R : RHeap} {r : Ref}
    (hC : Corr H R) (hx : H.Exclusive r) :
    checkExclusive R r = .ok r ∧
      ∀ r', checkExclusive R r ≠ .abort r'
```

```
theorem setAt_inPlace_of_entryInv
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hC : Corr H R) (hcell : H.cells root = some o)
    (hR : R.cells root = some o') (hin : i < o.fields.size) :
    ∃ H',
      setAt R root i (.scalar vn) = some (.inPlace H' root) ∧
      Corr (inPlaceOutcome root i vn o H).heap H'
```

Claim 11 (Emitted-semantics abort impossibility).

```
def PrefixAborts (out : Option PrefixOutcome) : Prop :=
  ∃ o, out = some o ∧ o.fault ≠ none

theorem prefix_abort_impossible
    (hproj : projectPrefix code = some p)
    (hEntry : EntryInv H env (Runner.initCtx root) root client)
    (hag : Agrees assign p s env H root i v)
    (hcell : H.cells root = some o) (hin : i < o.fields.size) :
    ¬ PrefixAborts (PrefixOutcome.ofExec assign p (execAST assign p code s))
```

Hypotheses abbreviated: the layout and the remaining entry premises.

8 Empirical alignment

The compiled corpus and the gate: the model's predicates against `lean_is_exclusive` and address identity, with the check armed and unarmed. Report the closed-pure-array finding and the differential corpus; state that this is validation, not an exhaustive equivalence. Cite the pinned artifact, IR-shape gate, generated C and runtime header.

Case	Predicted	Observed
unique array	exclusive, in place	agree
live alias (count two)	shared, copy	agree
alias released before the write	exclusive, in place	agree
persistent literal	shared, copy	agree
closure-held array	shared, copy	agree

9 Limits and non-claims

> **Non-claim 1.** *No C-semantics refinement; A10 replay/backend identity; the emitted recursive callee and continuation (N10); the flat-environment index bound; address identity; free/cascade; concurrency and callbacks; M7 source correspondence; the leaf body contract and the remaining heap-shape premises.*

10 Related work

> *Four clusters, one sentence each: reference counting and reuse [12, 17, 10]; linear and uniqueness disciplines [18, 1, 14, 15, 3, 5]; program logics and ownership [13, 7, 6, 9]; verified compilation and kernels [11, 8, 4, 2].*

11 Conclusion

> *Restate what is proved and what is trusted; sketch the next step (a verified runtime model or a stronger backend gate).*

A Theorem map and artifacts

> *Lean name, file, and status (proved, conditional, measured, trusted); artifact pins and gate commands.*

`check_sound`
`Ownership/Soundness.lean` — proved.

`checkN_sound_of_entryInv`
`Ownership/Entry.lean` — proved.

`project_sound`
`OwnershipLcnf/Semantics.lean` — proved under the primitive contracts.

`recognized_write_in_place`
`OwnershipLcnf/Semantics.lean` — proved under the primitive contracts.

`index_path_simulates_model`
`OwnershipLcnf/PathRecursion.lean` — proved; leaf contract and N10 remain.

`pair_accounting`
`Ownership/CountedPair.lean` — proved; restricted free.

`putBack_exclusive_of_wellFormed`
`Ownership/PutBack.lean` — proved; returned-handle accounting remains.

`checkExclusive_no_abort`
`OwnershipLcnf/RuntimeModel.lean` — proved in the runtime model.

`prefix_abort_impossible`
`OwnershipLcnf/NoAbort.lean` — proved under A6/A8.

`differential corpus`
`runtime-bridge` — measured.

References

[1] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: Practical linearity in a higher-order polymorphic language. In *Principles of Programming Languages (POPL)*, 2018.

[2] Mario Carneiro. Lean4lean: Towards a formalized proof checker for lean 4, 2024. Preprint.

[3] Dave Clarke, John Potter, and James Noble. Ownership types for flexible alias protection. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 1998.

[4] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE-28*, volume 12699 of *LNCS*. Springer, 2021.

[5] Trevor Jim, J. Gregory Morrisett, Dan Grossman, Michael W. Hicks, James Cheney, and Yanling Wang. Cyclone: A safe dialect of c. In *USENIX Annual Technical Conference*, 2002.

[6] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. Rustbelt: Securing the foundations of the rust programming language. In *Principles of Programming Languages (POPL)*, 2018.

[7] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, and Lars Birkedal. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In *Principles of Programming Languages (POPL)*, 2015.

[8] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. Cakeml: A verified implementation of ml. In *Principles of Programming Languages (POPL)*, 2014.

[9] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. In *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2023. Verify author list.

[10] Daan Leijen. Type directed compilation of row-typed algebraic effects. In *Principles of Programming Languages (POPL)*, 2017.

[11] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 2009.

[12] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. Perceus: Garbage free reference counting with reuse. In *Programming Language Design and Implementation (PLDI)*, 2021.

[13] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Logic in Computer Science (LICS)*, 2002.

[14] Sjaak Smetsers, Erik Barendsen, Marko van Eekelen, and Rinus Plasmeijer. Uniqueness typing for functional languages with graph rewriting semantics. *Mathematical Structures in Computer Science*, 1996. Verify volume and pages.

[15] Frederick Smith, David Walker, and J. Gregory Morrisett. Alias types. In *European Symposium on Programming (ESOP)*, 2000.

[16] The Lean FRO. Lean 4 source and runtime. `https://github.com/leanprover/lean4`. Pinned toolchain used in this work: Lean 4.35.0-rc3.

[17] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Implementation and Application of Functional Languages (IFL)*, 2019.

[18] Philip Wadler. Linear types can change the world! *Programming Concepts and Methods*, 1990.